HOUSEHUNT KENYA

Step-by-Step Product, Development and Technology Blueprint

A practical guide for building a trusted Kenyan house-hunting marketplace

Blueprint decision	Recommended starting position
Initial market	Kayole and surrounding Nairobi estates; expand after validating the model.
Initial property focus	Rental houses first. Design the data model so sales listings can be added later.
First product	Responsive website / Progressive Web App (PWA). Android app can follow.
Primary users	House seekers, landlords, property agents and platform administrators.
Core promise	Find genuine, currently available homes with clear prices, useful location information and direct contact.

Prepared as a build guide. Proposed prices, feature priorities and technology choices should be validated before launch.

1. Executive summary

HouseHunt Kenya is a digital property marketplace that helps people discover rental homes online. A seeker searches by estate, budget, house type, bedrooms and amenities; reviews photographs and listing details; then contacts the responsible landlord or agent or requests a viewing. Landlords and agents use dashboards to publish listings, update availability and respond to enquiries. Administrators moderate accounts and listings.

The first release should solve one problem reliably: helping a person find a genuine rental house that is still available. Do not begin by building every possible feature. Launch in a limited area, verify supply manually, measure user behaviour and improve the product before expanding.

2. Product scope and assumptions

Area	MVP decision	Later expansion
Property type	Long-term residential rentals	Property sales, commercial property and short stays
Geography	Kayole and nearby estates	Wider Nairobi, then other towns and counties
Platforms	Responsive web app / PWA	Dedicated Android app; iOS if justified
Contact	Phone and WhatsApp links; enquiry form	In-app messaging and richer communication tools
Payments	No rent collection in the first release	M-Pesa payments for subscriptions and promoted listings
Map	Estate/neighbourhood and approximate map pin	Advanced nearby search and travel-distance tools
Verification	Manual review and phone verification	Stronger identity/business verification workflows

Important: do not hold tenant deposits or rent on behalf of landlords in the MVP. This introduces financial, consumer-protection, dispute and operational responsibilities that should be assessed separately.

3. Users, roles and permissions

Role	Main actions	Restrictions / controls
Visitor / house seeker	Search, filter, view listing details, save favourites after registration, contact owner/agent, request viewing.	Cannot publish listings or access private owner information beyond approved contact details.
Landlord	Create and manage own properties, upload images, update rent and availability, respond to enquiries.	Can edit only listings they own or are authorised to manage.
Agent / property manager	Manage multiple authorised listings, handle enquiries and viewings, manage team profile where enabled.	Require clear ownership/management authorisation and account review.
Moderator	Review reported listings, request corrections, hide suspicious listings and record moderation decisions.	No access to payment credentials or unnecessary personal data.
Administrator	Manage users, roles, listings, categories, reports, platform settings and audit records.	Use strong authentication and log sensitive actions.

4. Step-by-step project plan

Step 1 — Validate the problem and local market

- Interview 15–25 house seekers in the launch area. Ask how they currently search, what information is missing, how often listings are unavailable and what makes them trust a listing.
- Interview 10–15 landlords or agents. Ask how they advertise, how they handle enquiries, whether they would maintain listings online and what they might pay for qualified leads.
- Audit competing property websites, Facebook groups and local agent practices. Record gaps rather than copying features.
- Create a small verified sample catalogue of 30–50 properties before public launch, with permission to publish photographs and contact details.

Deliverables: interview notes, competitor comparison, initial supply list, validated MVP requirements.

Step 2 — Define requirements and success measures

- Write user stories for seekers, landlords, agents and administrators. Example: 'As a seeker, I can filter rentals by estate and maximum rent so that I can shortlist suitable homes.'
- Define required listing fields, allowed values, validation rules, moderation states and what makes a listing 'available'.
- Set measurable MVP targets: verified listings, search-to-contact conversion, enquiry response time, stale-listing rate, viewing requests and repeat visits.
- Prioritise features as Must-have, Should-have, Could-have and Not-in-MVP.

Deliverables: Product Requirements Document (PRD), user stories, acceptance criteria and launch metrics.

Step 3 — Design the user experience

- Map the seeker journey: home page → search results → filters/map → property detail → contact or viewing request.
- Map the publisher journey: register → verify phone → create listing → upload images → submit for review → publish → update availability.
- Prepare low-fidelity wireframes before visual design. Test them with at least five potential users on mobile screens.
- Design for low bandwidth: compressed images, simple pages, clear loading states and minimal unnecessary animation.
- Make contact details and listing freshness visible without exposing private information unnecessarily.

Deliverables: sitemap, user-flow diagrams, mobile-first wireframes, design system and clickable prototype.

Step 4 — Choose and set up the technology

- Create a source-code repository and separate development, staging and production environments.
- Set up frontend, backend, database, image storage and automated deployment pipeline.
- Configure environment variables and secrets securely; never commit passwords, API keys or production credentials.
- Set up error monitoring, application logs, database backups and basic uptime checks before launch.

Deliverables: running development environment, repository, architecture diagram and deployment pipeline.

Step 5 — Build authentication and user profiles

- Implement registration, login, logout, password reset and phone-number verification.
- Use role-based permissions for seekers, landlords, agents, moderators and administrators.
- Create profile fields and account status: pending, active, restricted or suspended.
- Add rate limits and protections against repeated login attempts and automated account creation.

Deliverables: secure accounts, role permissions and account-management screens.

Step 6 — Build property listing management

- Create listing forms with title, description, rent, deposit if disclosed, property type, bedrooms, bathrooms, amenities, estate, approximate location and contact method.
- Allow multiple images, designate a cover image, validate file type and size, and compress images for web delivery.
- Implement listing lifecycle states: draft, submitted, under review, published, rejected, occupied, expired and archived.
- Record who created and last edited each listing. Require owners/agents to reconfirm availability at a defined interval.
- Prevent duplicate or misleading listings where possible using address/estate, contact, image and content checks plus moderator review.

Deliverables: publisher dashboard, listing workflow, image handling and moderation queue.

Step 7 — Build search, filters and location features

- Support keyword and structured search by county, town, estate, price range, bedrooms, property type and selected amenities.
- Sort by newest, price ascending and price descending. Clearly label promoted results.
- Add pagination or infinite scrolling and useful empty states when no matches are found.
- Store coordinates where available, but show approximate locations publicly if exact addresses create privacy or security concerns.
- Add map search after core text/filter search works. Ensure users can still search without map permissions.

Deliverables: fast search API, search-results page, filters and map-ready location data.

Step 8 — Build enquiries and viewing requests

- Provide click-to-call and WhatsApp contact actions, with the phone number controlled by the listing owner.
- Provide an enquiry form that records the property, seeker, message, date and status.
- Allow a seeker to request a viewing time; the landlord/agent can accept, decline or propose another time.
- Send email or SMS notifications where configured. Avoid sending repeated messages for the same event.
- Provide reporting and blocking flows for harassment, fraud or suspicious behaviour.

Deliverables: enquiry inbox, viewing-request workflow and notifications.

Step 9 — Build administration, trust and safety

- Create an admin dashboard for users, listings, reports, verification and moderation history.
- Require manual approval for new publishers or listings during the pilot.
- Add 'report listing' reasons such as already occupied, false price, copied images, suspected scam or incorrect location.
- Display listing status and 'last confirmed available' date. Expire listings that are not reconfirmed.
- Create a documented complaints process and escalation path. Keep an audit trail of administrative actions.

Deliverables: moderation tools, verification checklist, reporting process and audit logs.

Step 10 — Test the product

- Write unit tests for business rules and API permissions; integration tests for listing, search and enquiry workflows.
- Run end-to-end tests on common mobile and desktop browsers.
- Test slow networks, image failures, empty search results, invalid uploads, expired listings and simultaneous edits.
- Perform security checks for access control, input validation, file uploads, password handling and exposed personal data.
- Invite a small pilot group and fix critical issues before opening registration publicly.

Deliverables: test plan, defect register, security checklist and pilot sign-off.

Step 11 — Launch a controlled pilot

- Launch in the chosen estates with a verified set of properties and a small group of responsive landlords/agents.
- Monitor listing freshness, contact success, broken links, enquiry response times and complaints daily.
- Call or message a sample of seekers to learn whether listings were accurate and whether they successfully arranged viewings.
- Do not expand geography until supply quality and user trust are consistently acceptable.

Deliverables: pilot report, prioritised improvements and go/no-go decision for expansion.

Step 12 — Monetise and expand carefully

- Keep basic listings free initially to build supply. Test paid promotion only after the platform delivers useful enquiries.
- Add M-Pesa Daraja payments for platform fees, not tenant rent, unless a separate legal and operational review supports that service.
- Offer agent plans with transparent listing limits, analytics and support. Clearly mark sponsored listings.
- Expand to additional estates and towns based on demand, reliable listing supply and operational capacity.
- Consider a React Native Android application when mobile-web usage and retention justify the extra maintenance cost.

Deliverables: pricing experiment, payment integration, agent plans and expansion plan.

5. Recommended technology stack

Layer	Recommended technology	Purpose / notes
Web frontend	Next.js (React) + TypeScript	Responsive website, SEO-friendly public listing pages and reusable UI components.
UI styling	Tailwind CSS	Consistent responsive layouts and fast interface development.
Backend API	Python + Django + Django REST Framework	Business logic, authentication, permissions, listing workflows and REST API.
Database	PostgreSQL	Reliable relational database for users, properties, enquiries and transactions.
Geospatial	PostGIS	Estate coordinates, map pins and radius/nearby searches.
Image storage	Cloudinary or S3-compatible object storage	Store property photographs separately from the application server;

		generate optimised variants.
Maps	OpenStreetMap with Leaflet; evaluate Google Maps if needed	Map display and geocoding. Check provider terms, coverage, limits and pricing.
Search	PostgreSQL full-text search initially	Start simple; consider OpenSearch/Elasticsearch only when scale or relevance needs justify it.
Authentication	Django auth + secure session or token strategy	Password reset, roles and verified phone workflow. Use a maintained library for tokens if required.
Notifications	Email provider + Kenyan SMS gateway; WhatsApp Business Platform if approved	Enquiry alerts, verification codes and viewing updates. Check local delivery rates and consent rules.
Payments	Safaricom Daraja API	M-Pesa STK Push / payment confirmation for platform subscriptions and promotions.
Hosting	Managed cloud VM/app platform + managed PostgreSQL	Choose a provider with backups, monitoring, regional latency and clear scaling options.
CI/CD	GitHub + GitHub Actions	Run tests and deploy to staging/production after approved changes.
Monitoring	Sentry + provider logs/metrics	Track exceptions, slow endpoints, failed jobs and service health.
Mobile later	React Native + Expo	Share TypeScript skills and API contracts with web; build after the web MVP proves demand.

Alternative: a Django-rendered website can be even simpler for a small first team. The Next.js + Django API combination is recommended if the product is expected to grow into a separate mobile app.

6. High-level system architecture

Browser / PWA → Next.js frontend → Django REST API → PostgreSQL/PostGIS. The API also connects to object storage for images, an email/SMS provider for notifications, a maps provider for geocoding/map tiles and Safaricom Daraja for platform payments in a later phase.

Component	Responsibilities	Important design rule
Frontend	Pages, forms, search filters, dashboards and accessibility.	Do not trust browser-side validation alone; validate again on the server.
API/backend	Authentication, permissions, listing rules, enquiries, moderation and payment callbacks.	Enforce ownership and role checks on every protected endpoint.
Database	Persistent structured data and geographic coordinates.	Use migrations, indexes, constraints and scheduled backups.
Image storage	Property images and thumbnails.	Use signed or controlled upload flows, size limits and safe file validation.
Background jobs	Expiry reminders, availability reconfirmation, notifications and image processing.	Use a queue such as Celery + Redis only when asynchronous work is needed.
External services	Maps, SMS, email, WhatsApp and M-Pesa.	Use timeouts, retries, webhook verification and graceful failure handling.

7. Core database design

Table	Key fields	Purpose
users	id, name, email, phone, password_hash, role, status, phone_verified_at, created_at	Account identity and access role.
profiles	user_id, organisation_name, profile_photo, bio, verification_status	Additional landlord/agent profile details.
locations	id, county, town, estate, latitude, longitude, public_location_label	Structured location and geographic search.
properties	id, owner_id, location_id, title, description, property_type, rent_amount, bedrooms, bathrooms, status, published_at, last_confirmed_at	Main listing record.
property_images	id, property_id, storage_key, alt_text, sort_order, is_cover	Image references and gallery order.
amenities	id, name, category	Reusable amenity catalogue.
property_amenities	property_id, amenity_id	Many-to-many property amenities.
enquiries	id, property_id, seeker_id, message, contact_preference, status, created_at	Track expressions of interest.
viewing_requests	id, property_id, seeker_id, proposed_at, status, response_note	Arrange property viewings.
favourites	user_id, property_id, created_at	Saved properties.
reports	id, reporter_id, property_id, reason, details, status, resolved_by	Fraud and quality reports.
verification_records	id, user_id/property_id, method, status, reviewed_by, reviewed_at	Evidence and decision history.

subscriptions	id, user_id, plan_id, status, starts_at, ends_at	Agent plans and paid features in later phase.
payments	id, user_id, purpose, amount, provider_reference, status, created_at	Platform fee payment records; never store M-Pesa PINs.
audit_logs	id, actor_id, action, entity_type, entity_id, created_at	Trace sensitive administrative actions.

Use database foreign keys and indexes. Add indexes for property status, location, rent, bedrooms and publication date. Add a PostGIS geography/geometry index for coordinates. Store money as a decimal or integer minor units with an explicit currency (KES); do not use floating-point types for prices.

8. Main screens to design and build

Screen	Must contain	Primary action
Home / search	Location search, budget, property type, quick filters, recent/featured listings.	Search homes.
Search results	Result count, filters, sort, listing cards, map/list toggle, pagination.	Open a property.
Property detail	Image gallery, price, estate, features, availability confirmation, contact details and report link.	Call, WhatsApp or request viewing.
Register / login	Phone/email, password, consent and verification flow.	Create or access account.
Create listing	Guided form, location, rent, features, photos and preview.	Submit for review.
Publisher dashboard	Listings by status, enquiries, viewing requests and profile verification.	Manage a listing.
Enquiry inbox	Property, seeker message, timestamp and response status.	Respond to seeker.
Admin dashboard	Pending listings, reported listings, user status, verification and audit history.	Approve, reject or investigate.
Help / safety	How to avoid scams, reporting process, contact and privacy information.	Report a concern.

9. API design — initial endpoint plan

Method and endpoint	Purpose	Access
POST /api/auth/register	Create account	Public
POST /api/auth/login	Authenticate user	Public
POST /api/auth/verify-phone	Verify phone code	Authenticated / challenge
GET /api/properties	Search and filter published listings	Public
GET /api/properties/{id}	Read one published listing	Public
POST /api/properties	Create draft listing	Landlord / agent
PATCH /api/properties/{id}	Edit own listing	Owner / authorised agent
POST /api/properties/{id}/submit	Submit listing for review	Owner / authorised agent
POST /api/properties/{id}/images	Upload listing images	Owner / authorised agent
POST /api/properties/{id}/enquiries	Send an enquiry	Public or authenticated with abuse controls
POST /api/properties/{id}/viewings	Request a viewing	Authenticated seeker
GET /api/me/enquiries	View received/sent enquiries	Authenticated
POST /api/properties/{id}/reports	Report a listing	Public or authenticated with abuse controls
GET /api/admin/moderation	List items awaiting review	Moderator / admin
POST /api/admin/properties/{id}/decision	Approve, reject or suspend listing	Moderator / admin

Use consistent JSON responses, pagination, filtering parameters, clear error codes and API versioning. Document the API with OpenAPI/Swagger. The exact routes can change during implementation.

10. Security, privacy and trust requirements

- Use HTTPS everywhere and secure cookies/session settings. Store passwords using Django's supported password hashing.
- Apply role-based access control and object-level ownership checks. A landlord must not be able to edit another landlord's property by changing an ID.
- Validate and sanitise all input on the server. Protect against SQL injection, cross-site scripting, CSRF and insecure direct object references.

- Restrict image uploads by file size and allowed formats; inspect files and avoid serving user uploads as executable content.
- Keep API keys and payment credentials in environment secrets. Rotate credentials if exposed.
- Verify payment webhooks/callbacks and make payment processing idempotent so duplicate callbacks do not create duplicate payments.
- Collect only personal data required for the service, publish a clear privacy notice and define retention/deletion procedures.
- Review obligations under Kenya's Data Protection Act, 2019 and applicable ODPC guidance. Obtain appropriate legal advice before processing identity documents or sensitive verification data.
- Do not publish a landlord's private home address or a tenant's personal information without a clear lawful basis and appropriate consent.
- Provide a visible scam-awareness guide: never pay to view a house without appropriate checks, verify who is showing the property and confirm terms before sending money.

11. Quality assurance and acceptance criteria

Area	Minimum acceptance test
Search	A seeker can filter published properties by estate and rent range; occupied, rejected and expired listings are excluded.
Listing	A publisher can save a draft, upload valid images, submit for review and see the moderation outcome.
Permissions	A user cannot edit, delete or view private records belonging to another user without permission.
Availability	A publisher can mark a property occupied; it disappears from normal available search results.
Freshness	Listings not reconfirmed within the configured period are flagged and then expired or hidden according to policy.
Enquiry	A seeker can send an enquiry; the responsible publisher receives it and its status is recorded.
Mobile	Core search and contact journeys work on common Android phone screen sizes and low-bandwidth connections.
Security	Authentication, upload validation, access control and payment callback checks pass security tests.
Reliability	Backups are configured and a restore test is completed before public launch.

12. Suggested delivery team and responsibilities

Role	Responsibility	Can be combined in a small team?
Product owner	Priorities, user research, acceptance decisions and local partnerships.	Founder can perform this role.
UI/UX designer	Research, wireframes, visual design and usability testing.	Can be part-time or contracted.
Frontend developer	Next.js pages, responsive UI and API integration.	Can be combined with backend by a full-stack developer.
Backend developer	Django API, database, permissions, workflows and integrations.	Can be combined with frontend by a full-stack developer.
QA tester	Test cases, regression testing and pilot feedback.	Founder/team can start with a structured checklist.
Operations/moderation	Landlord onboarding, listing checks, complaints and stale-listing follow-up.	Essential even if part-time at pilot stage.

13. Indicative implementation sequence

A small experienced team might plan a 10–14 week MVP, but the schedule depends on team size, design readiness, integration approvals and how much listing verification is manual. These are planning estimates, not a delivery guarantee.

Workstream	Indicative duration	Exit condition
Discovery and requirements	1–2 weeks	Validated MVP, PRD and pilot area.
UX and technical setup	1–2 weeks	Approved wireframes and development environment.
Accounts and listing management	2–3 weeks	Users can register and submit listings.
Search and property details	2 weeks	Seekers can find and inspect available properties.
Enquiries, admin and moderation	1–2 weeks	Contact workflows and moderation work end-to-end.
Testing and pilot preparation	1–2 weeks	Critical bugs fixed; initial verified listings loaded.
Controlled pilot	2 weeks initially	Real user feedback and operational metrics collected.

14. Deployment and operations checklist

- Register a suitable domain name and configure DNS.
- Deploy the frontend and backend with separate staging and production configurations.
- Use managed PostgreSQL where possible; enable automated backups and test restoration.
- Configure image storage, CDN/image optimisation and upload limits.
- Enable HTTPS, security headers, error monitoring and uptime monitoring.
- Set up automated tests and controlled deployment from the main branch.
- Create a process for database migrations and rollback planning.
- Create an admin account with multi-factor authentication where supported.
- Publish Terms of Service, Privacy Notice, listing rules, prohibited content policy and complaint channel.
- Prepare customer-support scripts and a process for removing fraudulent or unavailable listings.

15. Cost categories to budget for

Actual costs depend on traffic, provider choice, image volume, SMS usage, maps usage and whether development is done by the founder, freelancers or an agency. Obtain current quotations before committing.

Cost category	What drives the cost	Cost-control approach
Domain and email	Domain renewal and business email accounts.	Start with one domain and a small number of mailboxes.
Hosting and database	CPU, memory, storage, backups and bandwidth.	Use modest managed hosting and monitor usage.
Image storage/CDN	Number and size of photos and image delivery traffic.	Compress, resize and limit images per listing.
Maps/geocoding	Map tiles, geocoding requests and usage tiers.	Start with open map data where suitable; monitor provider terms and usage.
SMS/WhatsApp/email	Verification messages and notifications sent.	Use SMS only where necessary; allow email/WhatsApp contact choices.
Development	Design, engineering, testing and maintenance hours.	Keep MVP scope narrow and prioritise the critical seeker journey.
Operations	Listing verification, customer support and agent onboarding.	Pilot in a compact area and create repeatable checklists.
Legal/compliance	Privacy review, terms and business setup.	Budget for professional review before handling sensitive data or payments.

16. Risks and mitigation

Risk	Why it matters	Mitigation
Fake or misleading listings	Damages trust and can expose seekers to fraud.	Manual checks, phone verification, reports, moderation and audit trail.
Outdated availability	Users waste time contacting owners for occupied homes.	Periodic reconfirmation, expiry rules and visible last-confirmed date.
Not enough listings	Search results look empty and users leave.	Secure supply before launch; onboard a small reliable landlord/agent group.
Slow or costly image delivery	Poor experience on mobile data.	Image compression, thumbnails, CDN and upload limits.
Low response from owners	Enquiries do not turn into viewings.	Response-time monitoring, reminders and publisher quality scoring.
Payment disputes	Can create financial and legal exposure.	Do not collect rent/deposits in MVP; use payment integration only for platform fees initially.
Scope creep	Delays launch and consumes budget.	Use Must/Should/Could prioritisation and protect MVP scope.

17. MVP launch checklist

- ☐ At least 30–50 permissioned and manually checked rental listings are ready.
- ☐ Landlord/agent accounts and contact details have been checked.
- ☐ Search works by estate, price and property type.

- ☐ Each listing has clear images, rent, location, features and availability status.
- ☐ Listing freshness policy and expiry automation are active.
- ☐ Users can contact the responsible publisher and report a listing.
- ☐ Admin moderation and account suspension work.
- ☐ Privacy notice, terms, listing rules and safety guidance are published.
- ☐ Mobile testing, security checks, backup and restore test are complete.
- ☐ Support contact and issue-response process are ready.

18. Recommended immediate next actions

1. Confirm the working name, launch estates and rental-only MVP scope.
2. Conduct the first interviews with seekers, landlords and agents.
3. Prepare a PRD and screen-by-screen wireframes from this blueprint.
4. Create the GitHub repository and technical project skeleton.
5. Build the first vertical slice: publish one listing, search for it, open its detail page and contact its publisher.
6. Use that working slice to test the complete value proposition before adding advanced features.

19. Useful official technology references

- **Django documentation:** https://docs.djangoproject.com/
- **Django REST Framework:** https://www.django-rest-framework.org/
- **Next.js documentation:** https://nextjs.org/docs
- **PostgreSQL documentation:** https://www.postgresql.org/docs/
- **PostGIS documentation:** https://postgis.net/documentation/
- **React Native documentation:** https://reactnative.dev/docs/getting-started
- **Safaricom Daraja:** https://developer.safaricom.co.ke/
- **OpenStreetMap:** https://www.openstreetmap.org/
- **Leaflet:** https://leafletjs.com/
- **Kenya ODPC:** https://www.odpc.go.ke/

20. Final recommendation

Build a focused, mobile-first rental discovery website for one compact Nairobi market. Make listing accuracy, availability freshness and reliable contact the product's defining qualities. Use Next.js, Django REST Framework, PostgreSQL/PostGIS and managed image storage as the proposed stack. Keep payments, native mobile apps, sales listings and complex messaging outside the first release. Prove that seekers can find real homes and landlords receive useful enquiries; then invest in expansion.